

Structural Design of Convolutional Neural Networks for Steganalysis

Guanshuo Xu, Han-Zhou Wu, *Student Member, IEEE*, Yun-Qing Shi, *Fellow, IEEE*

Abstract—Recent studies have indicated that the architectures of convolutional neural networks (CNNs) tailored for computer vision may not be best suited to image steganalysis. In this letter, we report a CNN architecture that takes into account knowledge of steganalysis. In the detailed architecture, we take absolute values of elements in the feature maps generated from the first convolutional layer to facilitate and improve statistical modeling in the subsequent layers; to prevent overfitting, we constrain the range of data values with the saturation regions of *TanH* at early stages of the networks, and reduce the strength of modeling by using 1×1 convolutions in deeper layers. Although it learns from only one type of noise residual, the proposed CNN is competitive in terms of detection performance compared with the SRM with ensemble classifiers on the BOSSbase for detecting S-UNIWARD and HILL. The results have implied that well-designed CNNs have the potential to provide a better detection performance in the future.

Index Terms—Convolutional neural networks (CNN), deep learning, steganalysis, forensics.

I. INTRODUCTION

THE feature design in conventional machine learning based steganalysis [1-6] could be suboptimal when the data embedding methods are effective and sophisticated [7-12]. Strong in feature learning, the convolutional neural networks (CNNs) [13, 14] have made tremendous achievements in computer vision since 2012 [14-18]. This has aroused the interest of researchers in the field of information forensics, in particular, on steganalysis, in seeking the way to use CNNs for steganalysis [19-21, 26].

In their pioneering work, Tan and Li [20] proposed a CNN which comprises three stages of alternating convolutional and max pooling layers, and sigmoid non-linear activations. When detecting HUGO [7] at embedding rate of 0.4 bits per pixel (bpp) on the BOSSbase [22], the CNN achieved the error rate of 48% with random parameter initialization; after involving a high-pass kernel [3, 23] into parameter initialization of the first

convolutional layer, and pre-training all of the parameters with unsupervised learning, they managed to reduce the error rate to 31%, still far away from the error rate of 14% achieved by the SRM [3] with ensemble classifiers [25].

A few months later, Qian *et al.* [21] presented a CNN equipped with a high-pass filtering layer, Gaussian non-linear activations, and average pooling for steganalysis. The reported detection error rates are 2% to 5% higher than those achieved by the SRM on the BOSSbase with images downsized from 512×512 to 256×256 , when detecting HUGO [7], WOW [8], and S-UNIWARD [9]. This is a significant boost in performance, but it is still inferior to the SRM.

Studies in these two pieces of works have indicated that taking into account the (domain) knowledge in steganography and steganalysis (e.g., using high-pass kernels to generate noise residuals) improves the detection performance of the CNNs. In conventional steganalysis, the domain knowledge is embedded in the feature extraction. Analogously, as the CNNs embrace the feature extraction step into the networks, the domain knowledge should be reflected in the network architectures.

Along this direction, in this letter, we propose a CNN that tries to incorporate the knowledge of steganalysis. In the detailed architecture, we take absolute values of elements in the feature maps generated from the first convolutional layer to facilitate and improve statistical modeling in the subsequent layers; to prevent overfitting, we constrain the range of element values at early stages of the networks, and reduce strength of modeling by using 1×1 convolutions in deeper layers; besides, as have been proved effective in the previous works [20, 21, 26], the proposed CNN learns from noise residuals and uses average pooling. Although the proposed CNN is neither large nor deep, and currently learns from only one type of noise residual, the experimental results have verified that its performance is comparable with that of the SRM with ensemble classifiers. This initial-stage work has confirmed that deep learning with CNNs is indeed a powerful machine learning tool for steganalysis. The results have also implied that a well-designed CNN would have the potential to provide a better detection performance when compared with the conventional machine learning based steganalysis.

II. THE PROPOSED CNN

In this section, the overall architecture of the proposed CNN is introduced first. Then, the design in the CNN to improve statistical modeling is presented. Finally, we discuss about our design considerations on the non-linear activations and spatial

Copyright (c) 2015 IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

This work is sponsored by NJIT Faculty Seed Grant Initiative.

G. Xu and Y. Q. Shi are with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: gx3@njit.edu; shi@njit.edu).

H. Wu is with Southwest Jiaotong University, Chengdu 611756 China, and is currently a visiting scholar at New Jersey Institute of Technology (e-mail: h.wu.phd@ieee.org).

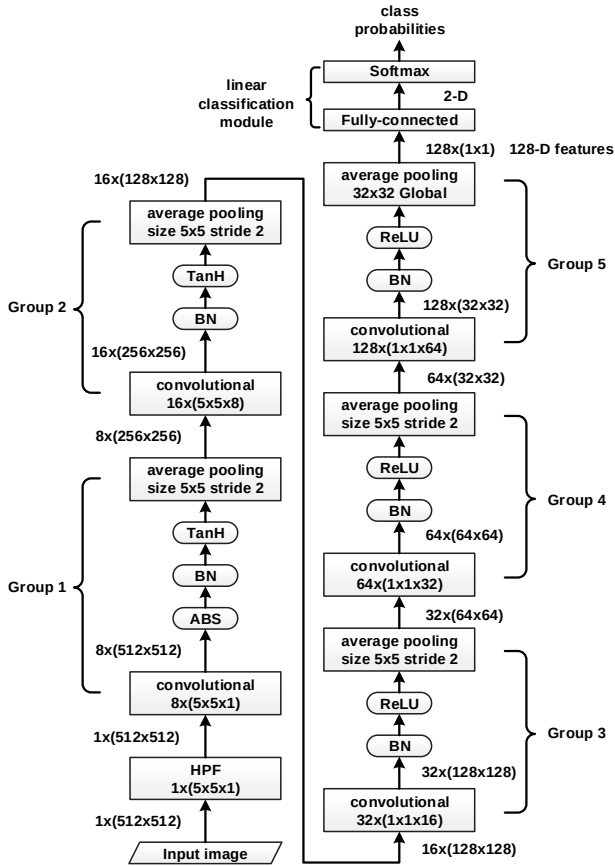


Fig. 1. The proposed CNN architecture. Layer types and parameter sizes (choices) are displayed inside boxes. Data sizes are displayed on the two sides. Sizes of convolution kernels in the boxes follow (number of kernels) $\times$ (height $\times$ width $\times$ number of input feature maps). Sizes of data follow (number of feature maps) $\times$ (height $\times$ width). Padding is applied in convolutional and pooling layers as well as the HPF layer.

sizes of convolutional kernels.

A. Overall Architecture

Fig. 1 illustrates the overall architecture of our CNN. A high-pass filtering (HPF) layer with the same kernel employed in [21] and [26], whose parameters are not optimized during training, is placed at the very beginning to transform original images to noise residuals in order to boost the signal to noise/interference ratio.

The whole CNN can be divided into a convolutional module which transforms the images to feature vectors (128-D), and a linear classification module (composed of a fully-connected layer and a softmax layer) which transforms feature vectors to output probabilities for each class.

The convolutional module comprises five groups of layers (displayed as ‘Group 1’ to ‘Group 5’ in Fig. 1), each starts with a convolutional layer which generates feature maps, and ends with an average pooling layer which performs local averaging as well as subsampling on the feature maps (except Group 5).

To enhance the power of statistical modeling, our CNN is equipped with the hyperbolic tangent (TanH) (Fig. 3. (a)) non-linear activations for Group 1 and Group 2, and the rectified linear unit (ReLU) [24] activations (Fig. 3. (b)) for

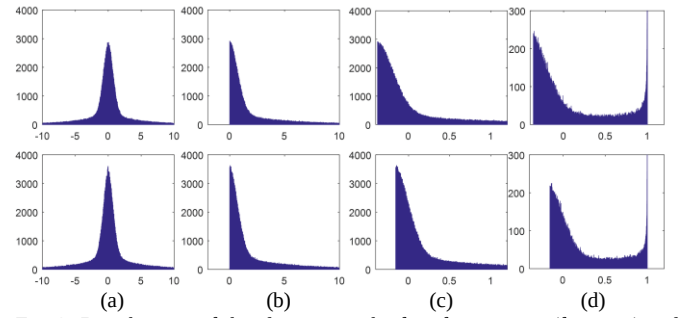


Fig. 2. Distributions of the elements in the first feature map (first row) and those in the second feature map (second row) after going through (a) convolutional layer, (b) ABS layer, (c) BN layer, (d) TanH layer, in Group 1.

TABLE I
THE OPTIMIZED SCALES AND BIASES (EQUATION 2) IN THE BN LAYER IN GROUP 1 AFTER THE NORMALIZATION STEP (EQUATION 1)

k	1	2	3	4	5	6	7	8
γ^*	1.20	1.24	1.15	1.61	0.91	0.93	1.04	1.10
β^*	0.29	0.51	0.50	0.55	0.25	0.33	0.58	0.40

Group 3, Group 4, and Group 5. Inside Group 1, an absolute activation (ABS) layer is inserted to force the statistical modeling to take into account the (sign) symmetry [3] existed in noise residuals. To prevent the CNN training from falling into poor local minima, and learn optimal scales and biases for feature maps, batch-normalization (BN) [17] is performed immediately before each non-linear activation layer.

Finally, through global averaging, the pooling layer in Group 5 merges each spatial map to a single element (128 maps of size 32×32 to 128-D features). In this way, the whole CNN is constraint to perform the same operations to every pixel in the original images (or the noise residuals), thereby preventing the statistical modeling from grasping the location information of embedded pixels from the training data. Note that the recent work [26] addresses the steganalysis scenario when the embedding key is reused for all the images, resulting in leakage of embedding locations. Hence, it differs in methodology with the work reported here.

B. Improving Statistical Modeling

The exact modeling procedure in the CNN is hard to interpret when the layer-wise computation goes deeply. Therefore, we stand a better chance to improve the performance by focusing more the design of the first layer group (Group 1).

Group 1 starts with a convolutional layer that takes as input the noise residuals generated from the HPF layer. This convolutional layer explores relations of neighboring pixels in the residuals, and generates feature maps for statistical modeling. To assist the statistical modeling of the CNN, we disable the default bias learning in this convolutional layer so that the feature maps are symmetric with respect to zeros (sign-symmetry [3]); then, we insert an ABS layer right after this convolutional layer to discard the signs of the elements in the feature maps. The output of the ABS layer is thereafter fed into a BN layer. The BN layer first normalizes elements in each input feature map to zero-mean and unit-variance to ensure that the initial input to TanH falls in the quasi-linear region, shown in Fig.3 (a), and hence the gradient back-propagation would not

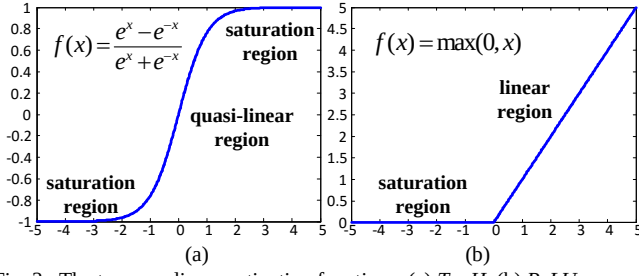


Fig. 3. The two non-linear activation functions: (a) *TanH*, (b) *ReLU*

fall into poor local minima. Data in the convolutional module of the CNN have four dimensions: (n, k, i, j) , n denotes the n -th input data in the mini-batch in the training stage ($1 \leq n \leq N$), k denotes the k -th feature map ($1 \leq k \leq K$), i and j ($1 \leq i \leq H$, $1 \leq j \leq W$) are the spatial height and width indices in the feature maps. Let $x_{n,i,j}^{(k)}$ and $\hat{x}_{n,i,j}^{(k)}$ represent, respectively, the input elements of the k -th feature map and the corresponding normalized feature map, the normalization step for the k -th feature maps can be written as

$$\hat{x}_{n,i,j}^{(k)} = \frac{x_{n,i,j}^{(k)} - \mu^{(k)}}{\sigma^{(k)}}, \quad (1)$$

where

$$\mu^{(k)} = \frac{1}{NHW} \sum_{n,i,j} x_{n,i,j}^{(k)}, \quad \sigma^{(k)} = \sqrt{\frac{1}{NHW} \sum_{n,i,j} (x_{n,i,j}^{(k)} - \mu^{(k)})^2}.$$

To recover the power of modeling, the BN layer then scales and shifts the normalized data with a scaling factor $\gamma^{(k)}$ and a bias $\beta^{(k)}$, both optimized, for each normalized feature map, and generates an output $y_{n,i,j}^{(k)}$ which can be calculated by

$$y_{n,i,j}^{(k)} = \gamma^{(k)} \hat{x}_{n,i,j}^{(k)} + \beta^{(k)}. \quad (2)$$

As all the normalization, scaling and shifting processing in the BN layer are identical within each feature map (demonstrated in Equation (1)), the spatial correlations between elements are well preserved; contrastively, as those processing differ across feature maps, optimal scaling and shifting parameters could be learned for each feature map. Because of the bias terms introduced in the BN layer, without the ABS layer, the sign-symmetry of elements in the output feature maps of the first convolutional layer would no longer hold, causing interference between feature values at early stages of statistical modeling. This problem could be solved once the sign information is discarded by the ABS layer.

To provide some intuitive understanding of the functionality in Group 1, we drawn in Fig. 2 the distributions of a test image after it went through the convolutional layer, ABS layer, BN layer, and *TanH* activation in Group 1 of a trained CNN (only the first two feature maps are displayed). Since we disabled bias learning in the first convolutional layer, the output distributions, in Fig. 2 (a), are symmetric with respect to zeros. The output of the ABS layer, with the distributions shown in Fig. 2 (b), are

first normalized following Equation (1) with the global statistics stored during training instead of statistics obtained from the input mini-batch. The normalized feature maps are then scaled and shifted (with biases) for optimal statistical modeling, the distributions after this step are displayed in Fig. 2 (c). Table I records the optimized scaling and shifting values for all the eight feature maps in the first BN layer. The output of the BN layer are then mapped and bounded by the *TanH* activation, as shown in Fig.2 (d).

C. Other Design Considerations

In this section, we describe our considerations on two other parts of the CNN design: the non-linear activations and spatial sizes of convolutional kernels.

In our CNN designed for steganalysis, a hybrid of *TanH* and *ReLU* non-linear activations are employed. We discovered that using the *TanH* instead of the more popularly used *ReLU* as non-linear activations in Group 1 and Group 2 improves performance. This could be attributed to the saturation regions of the *TanH*, which effectively limit the range of data values (illustrated in Fig.3 (a) and Fig.2 (d)) and prevent the deeper layers from modeling large values, as they are usually sparse and statistically not significant. It is also discovered that results became worse when more *ReLU*s in deeper layers were replaced by the *TanH*, likely due to the difficulty of gradient back-propagation with *TanH* [27].

Conventional machine learning based steganalysis models patterns of correlations in a small local region of residual elements, and adopts one-shot histogram pooling to prevent modeling larger regions [1-6]. In contrast, the CNN works by modeling (with alternating convolutional and pooling layers) relationships of residual elements over the whole images. On the one hand, such type of modeling is the root of the strength of the CNN, on the other hand, without effective control, overfitting (learning more image content than stego noise, and/or fitting the stego noise in training data too well to generalize to testing data) could occur for steganalysis. Having realized this potential issue, in the proposed CNN, we limit the sizes of convolutional kernels in deeper layers, i.e., the spatial sizes of the convolutional layers in the last three groups are limited to 1×1 so as to constrain the strength of modeling. Note that the selection of 1×1 convolution should not be taken for granted. It is expected that with more training data larger convolutional sizes would be preferred.

III. EXPERIMENTS

A. Dataset and Software Platforms

We performed experiments using our designed CNN to detect two spatial domain content-adaptive steganographic algorithms: S-UNIWARD [9] and HILL [11], with embedding rates of 0.1 bpp and 0.4 bpp. The corresponding performance achieved by the SRM [3] with ensemble classifier [25] is used as references. All of the experiments using the CNN were performed on a modified version of Caffe toolbox [28].

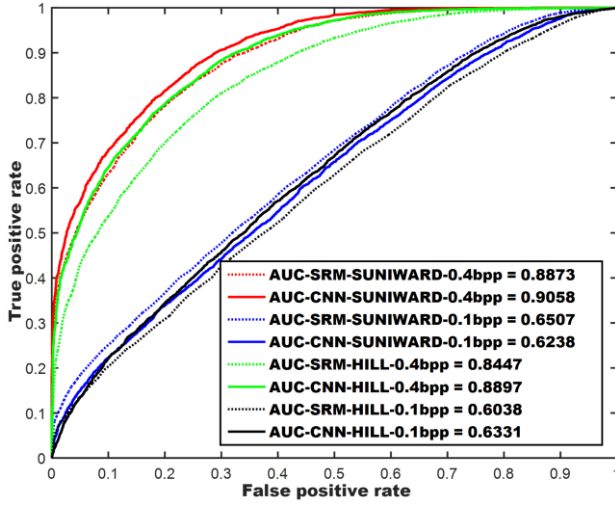


Fig. 4. ROC curves and their corresponding AUC illustrating the overall performance of CNN and SRM on detecting S-UNIWARD and HILL.

The dataset used is BOSSbase v1.01 [22] containing 10,000 cover images of size 512×512 . Image data of the other class (stego) were generated through data embedding into the cover images. To avoid confusion during performance comparison in the future works, we used the original image size (512×512) in all the experiments.

B. Training, Validation and Testing

For each experiment (a reported accuracy in Table II), out of the 10,000 pairs of images, 5,000 pairs were set aside for testing to verify the performance. These 5,000 testing pairs were not touched in the whole training phase.

In the training phase, we obtain totally five optimized CNN models from the 5,000 training pairs. Each of the CNN was trained on 4,000 pairs, and validated on 1,000 pairs. The validation pairs were used to decide when to stop training of each CNN. Specifically, we generated five 4,000/1,000 splits from the 5,000 training pairs by evenly breaking the 5,000 training pairs into five non-overlapping folds, and each time used a different fold for validation and the rest four folds for training. Note that this is not cross-validation, it is more like a controlled bootstrap aggregation [30] to create ensemble.

In the testing stage, the 10,000 testing images (5,000 testing pairs) went through all the five trained CNNs one by one, and the five output probabilities were averaged for each test image to make the final prediction.

C. Hyperparameters

Mini-batch gradient descent was used to train all the CNNs in experiments. The momentum was fixed to 0.9. The learning rate was initialized to 0.001, and scheduled to decrease 10% for every 5,000 iterations, for all the parameters. A mini-batch of 64 images (32 cover/stego pairs) was input for each iteration. All of the CNNs were trained for 120,000 iterations. Parameters in convolution kernels were initialized by random numbers generated from zero-mean Gaussian distribution with standard deviation of 0.01; bias learning were disabled in convolutional layers and fulfilled in BN layers. Parameters in the last fully-connected (FC) layers were initialized using ‘Xavier’ initialization [27]. Except for the FC layer, weight decay (L2 regularization) was not enabled so that we could focus on

TABLE II
ACCURACY (%) OF CNN AND SRM AGAINST S-UNIWARD AND HILL

	0.1 bpp		0.4 bpp	
	CNN	SRM	CNN	SRM
S-UNIWARD	57.33	59.25	80.24	79.53
HILL	58.44	56.44	79.24	75.47

TABLE III
MEANS AND STD OF SINGLE CNN MODEL ACCURACIES (%)

	SUNIWARD		HILL	
	0.1 bpp	0.4 bpp	0.1 bpp	0.4 bpp
mean	56.85	79.03	57.55	77.58
std	0.91	0.67	0.68	0.30

TABLE IV
LOSS IN ACCURACY (%) WITH DIFFERENT CNN STRUCTURES

	3×3	5×5	wo-abs	rp-tanh
	0.47	1.34	3.30	2.24

3×3 = replace all 1×1 convolution kernels with 3×3 kernels

5×5 = replace all 1×1 convolution kernels with 5×5 kernels

wo-abs = without the absolute layer

rp-tanh = replace the *TanH* layer in Group 2 with *ReLU*

designing the CNN architecture. For the same reason, no ‘dropout’ [29] was used.

D. Results

The experiments using the SRM (with ensemble classifiers [25]) were conducted on the same 5,000/5,000 train/test split as for the CNNs. To evaluate the performance, we used the average accuracies, recorded in Table II, and the receiver operating characteristic (ROC) curves together with the corresponding area under ROC curves (AUC), illustrated in Fig. 4. Overall, the CNN has competitive performance compared with the SRM and ensemble classifier. Table III reports the means and standard deviations of the testing accuracy obtained from the five single CNN models.

In Table IV, we record the losses in validation accuracies by adjusting the structures of the CNN to support our design in Section II (B) and Section II (C). The baseline CNN is one of the single model in Table III to detect S-UNIWARD at 0.4 bpp. These results verified the effectiveness of our proposed design. In particular, adding the absolute layer to enhance statistical modeling and using the *TanH* activation after the first two convolutional layers, have resulted in large improvements on the detection performance.

IV. CONCLUSION

In this letter, it is shown that a well-designed CNN is a good steganalytic tool, and would have the potential, in the future, to provide a better detection performance compared with that achieved by the conventional feature based steganalysis. Currently the performance of the proposed CNN still could not compare with some more advanced steganalytic methods [31-34], which rely on knowing the probability maps generated during data embedding. It would be interesting future works to feed the CNN with, e.g., the probability maps of embedding, to further enhance the performance against content-adaptive steganography. Also, how to apply the CNN in the best way to defeat steganography in the JPEG domain [35-37] would be another important future work.

REFERENCES

- [1] D. Zou, Y. Q. Shi, W. Su and G. Xuan, "Steganalysis based on Markov Model of Thresholded Prediction-Error Image," in *Proc. IEEE Int. Conf. Multimedia and Expo, Toronto, Canada*, Jul. 9–12, 2006, pp. 1365–1368.
- [2] Y. Q. Shi, P. Sutthiwan and L. Chen, "Textural features for steganalysis," in *Proc. 14th Int. Conf. Inf. Hiding*, vol. 7692. Berkeley, CA, USA, May 2012, pp. 63–77.
- [3] J. Fridrich and J. Kodovský, "Rich models for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 3, pp. 868–882, Jun. 2012.
- [4] T. Pevný, P. Bas, and J. Fridrich, "Steganalysis by subtractive pixel adjacency matrix," *IEEE Trans. Inf. Forensics Security*, vol. 5, no. 2, pp. 215–224, Jun. 2010.
- [5] V. Holub and J. Fridrich, "Random projections of residuals for digital image steganalysis," *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 12, pp. 1996–2006, Dec. 2013.
- [6] L. Chen, Y. Q. Shi and P. Sutthiwan, "Variable multi-dimensional co-occurrence histogram for steganalysis," in *Proc. IWDW*, vol. 9023, pp. 559–573, Oct. 2014.
- [7] T. Pevný, T. Filler, and P. Bas, "Using high-dimensional image models to perform highly undetectable steganography," in *Proc. 12th Int. Workshop Information Hiding*, R. Böhme and R. Safavi-Naini, Eds., Calgary, Canada, Jun. 28–30, 2010, vol. 6387, Lecture Notes in Computer Science, pp. 161–177, Springer-Verlag, New York.
- [8] V. Holub and J. Fridrich, "Designing steganographic distortion using directional filters," in *Proc. 4th IEEE Int. Workshop Inf. Forensics Security*, Dec. 2012, pp. 234–239.
- [9] V. Holub, J. Fridrich, and T. Denemark, "Universal distortion function for steganography in an arbitrary domain," *EURASIP Journal on Information Security*, vol. 2014, no. 1, pp. 1–13, 2014.
- [10] V. Sedighi, R. Cogranne and J. Fridrich, "Content-Adaptive Steganography by Minimizing Statistical Detectability," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 2, pp. 221–234, Feb. 2016.
- [11] B. Li, M. Wang, J. Huang, and X. Li, "A new cost function for spatial image steganography," in *Proc. IEEE ICIP*, Paris, France, Oct. 2014, pp. 4206–4210.
- [12] B. Li, S. Tan, M. Wang and J. Huang, "Investigation on Cost Assignment in Spatial Image Steganography," *IEEE Trans. Inf. Forensics Security*, vol. 9, no. 8, pp. 1264–1277, Aug. 2014.
- [13] Y. Le Cun, B. Boser, J. S. Denker, D. Henderson, R. Howard, W. Hubbard, and L. Jackel, "Handwritten digit recognition with a backpropagation neural network," in *Advances in Neural Information Processing Systems 2*, D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1990, pp. 396–404.
- [14] A. Krizhevsky, I. Sutskever, and G. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1106–1114.
- [15] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg and L. Fei-Fei, "ImageNet large scale visual recognition challenge," *Int. J. Comp. Visual*, vol. 115, pp. 211–252, 2015.
- [16] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke and A. Rabinovich, "Going deeper with convolutions," in *Proc. IEEE Int. Conf. Computer Vision and Pattern Recognition*, pp.1–9, Jun. 2015.
- [17] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv: 1502.03167*, 2015.
- [18] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," *arXiv: 1512.03385*, 2015.
- [19] J. Chen, X. Kang, Y. Liu and Z. J. Wang, "Median Filtering Forensics Based on Convolutional Neural Networks," *IEEE Signal Process. Lett.*, vol. 22, no. 11, pp. 1849–1853, Nov. 2015.
- [20] S. Tan and B. Li, "Stacked convolutional auto-encoders for steganalysis of digital images," in *Proc. APSIPA*, pp. 1–4, Dec. 2014.
- [21] Y. Qian, J. Dong, W. Wang and T. Tan, "Deep learning for steganalysis via convolutional neural networks," in *Proc. SPIE*, vol. 9409, 2015.
- [22] P. Bas, T. Filler and T. Pevný, "Break our steganographic system – the ins and outs of organizing BOSS," in: *Proc. Inf. Hiding*, vol. 6958, pp. 59–70, May, 2011.
- [23] A. D. Ker and R. Böhme, E. J. Delp and P. W. Wong, Eds., "Revisiting weighted stego-image steganalysis," in *Proc. SPIE, Electronic Imaging, Security, Forensics, Steganography, and Watermarking of Multimedia Contents X*, San Jose, CA, Jan. 27–31, 2008, vol. 6819, pp. 5 1–5 17.
- [24] V. Nair and G.E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," *Proc. Int'l Conf. Machine Learning*, pp. 807–814, 2010.
- [25] J. Kodovsky, J. Fridrich and V. Holub, "Ensemble Classifiers for Steganalysis of Digital Media," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 2, pp. 432–444, Apr. 2012.
- [26] L. Pibre, J. Pasquet, D. Ienco and M. Chaumont, "Deep learning is a good steganalysis tool when embedding key is reused for different images, even if there is a cover source mismatch," in *Proc. SPIE*, pp. 14–18 Feb. 2016.
- [27] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. AISTATS*, 2010, pp. 249–256.
- [28] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proc. ACM Int. Conf. Multimedia*, pp. 675–678, 2014.
- [29] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, Jan. 2014.
- [30] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [31] T. Denemark, V. Sedighi, V. Holub, R. Cogranne and J. Fridrich, "Selection-channel-aware rich model for steganalysis of digital images," *IEEE Int. Inf. Forensics Security (WIFS)*, pp. 48–53, 2014.
- [32] W. Tang, H. Li, W. Luo, and J. Huang, "Adaptive steganalysis against WOW embedding algorithm," in *Proc. ACM Workshop Inf. Hiding Multimedia Secur. (IH&MMSec)*, pp. 91–96, Jun. 2014.
- [33] W. Tang, H. Li, W. Luo and J. Huang, "Adaptive Steganalysis Based on Embedding Probabilities of Pixels," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 4, pp. 734–745, Apr. 2016.
- [34] T. Denemark, J. Fridrich and P. Comesaña-Alfaro, "Improving Selection-Channel-Aware Steganalysis Features," in *Proc. SPIE*, pp. 14–18, Feb. 2016.
- [35] V. Holub and J. Fridrich, "Low-Complexity Features for JPEG Steganalysis Using Undecimated DCT," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 2, pp. 219–228, Feb. 2015.
- [36] J. Kodovsky and J. Fridrich, "Steganalysis of JPEG images using rich models," in *Proc. SPIE*, vol. 8303, pp. 83030A–13, Feb. 2012.
- [37] X. Song, F. Liu, C. Yang, X. Luo and Y. Zhang, "Steganalysis of Adaptive JPEG Steganography Using 2D Gabor Filters," in *ACM Workshop Inf. Hiding Multimedia Secur. (IH&MMSec)*, pp. 15–23, 2015.